DeepSeek V4.1 Flash model structure, and the Plan B layout on this machine

40 layers · hidden size 5,120 · 4 hyper-connection residual streams · vocabulary 129,280 · context 1M (YaRN 16×, native 64K) · 763B parameters in total

Layers

Left bars: where the dense part / routed experts are: GPU0 GPU1 Host memory

Token embedding 129,280 × 5,120 (BF16) → copied into 4 residual streams; every layer then reads and writes all 4

Encoder: layers 0–19

Computed for every prompt token; each layer = attention + MoE (6 of 384 experts, plus 1 shared expert)

L0	Sliding-window attention: last 128 tokens	
L1	Sliding-window attention: last 128 tokens	Engram
L2	Ratio 2 · writes compressed KV and index keys · picks its own top-512	KV src, Idx src
L3	Ratio 2 · reads L2's compressed KV and L2's top-512	
L4	Ratio 2 · reads L2's compressed KV and L2's top-512	
L5	Ratio 2 · reads L2's compressed KV and L2's top-512	
L6	Ratio 2 · reads L2's compressed KV and L2's top-512	
L7	Ratio 2 · reads L2's compressed KV and L2's top-512	
L8	Ratio 2 · writes compressed KV and index keys · picks its own top-512	KV src, Idx src
L9	Ratio 2 · reads L8's compressed KV and L8's top-512	
L10	Ratio 2 · reads L8's compressed KV and L8's top-512	
L11	Ratio 2 · reads L8's compressed KV and L8's top-512	
L12	Ratio 2 · reads L8's compressed KV and L8's top-512	
L13	Ratio 2 · reads L8's compressed KV and L8's top-512	
L14	Ratio 2 · writes compressed KV and index keys · picks its own top-512	KV src, Idx src, Engram
L15	Ratio 2 · reads L14's compressed KV and L14's top-512	
L16	Ratio 2 · reads L14's compressed KV and L14's top-512	
L17	Ratio 2 · reads L14's compressed KV and L14's top-512	
L18	Ratio 2 · reads L14's compressed KV and L14's top-512	
L19	Ratio 2 · reads L14's compressed KV and L14's top-512	

Decoder: layers 20–39

CED: prefill computes only tokens that later tokens can still see via the sliding window; fewer in later layers, only the last in L39

L20	Ratio 1 · writes compressed KV shared by L20–39 · picks top-512 + blocks	KV src, Idx src, Blocks
L21	Ratio 1 · reads L20's compressed KV and L20's top-512	
L22	Ratio 1 · reads L20's compressed KV and L20's top-512	
L23	Ratio 1 · reads L20's compressed KV and L20's top-512	
L24	Ratio 1 · reads L20's compressed KV · reselects top-512 in candidate blocks	Idx src
L25	Ratio 1 · reads L20's compressed KV and L24's top-512	
L26	Ratio 1 · reads L20's compressed KV and L24's top-512	
L27	Ratio 1 · reads L20's compressed KV and L24's top-512	
L28	Ratio 1 · reads L20's compressed KV · reselects top-512 in candidate blocks	Idx src
L29	Ratio 1 · reads L20's compressed KV and L28's top-512	
L30	Ratio 1 · reads L20's compressed KV and L28's top-512	
L31	Ratio 1 · reads L20's compressed KV and L28's top-512	
L32	Ratio 1 · reads L20's compressed KV · reselects top-512 in candidate blocks	Idx src
L33	Ratio 1 · reads L20's compressed KV and L32's top-512	
L34	Ratio 1 · reads L20's compressed KV and L32's top-512	
L35	Ratio 1 · reads L20's compressed KV and L32's top-512	
L36	Ratio 1 · reads L20's compressed KV · reselects top-512 in candidate blocks	Idx src
L37	Ratio 1 · reads L20's compressed KV and L36's top-512	
L38	Ratio 1 · reads L20's compressed KV and L36's top-512	
L39	Ratio 1 · reads L20's compressed KV and L36's top-512	

Final RMSNorm → output head 5,120 × 129,280 (BF16) → logits for the next token

DSpark draft model: 3 layers × 128 experts (MXFP4, 7.4 GiB) for speculative decoding; tried here, not adopted

Inside one layer (a decoder layer as an example)

4 residual streams · hyper-connections mix 4 into 1; weights Sinkhorn-normalized (20 iterations)

Attention

After RMSNorm; 64 query heads, only 1 KV head (K = V, 512 dims)

Query

x → q_a 5,120→1,280 (low-rank) → norm
→ q_b → 64 heads × 512 dims
RoPE (YaRN) on the last 64 dims
q_a's output also feeds the sparse indexer

Key/value

Sliding window: raw KV of last 128 tokens
Compressed KV: one row per r tokens
(r = 2 or 1, written at KV-source layers)
Indexer lets only the top-512 rows through

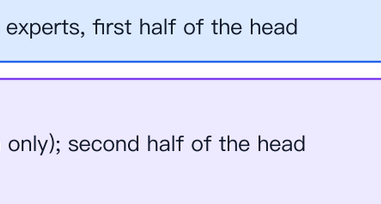
Attention computation

flash attention: 128 window rows + 512 compressed rows, one sink per head
Output projection: 8 low-rank groups (rank 1,024) → 5,120, back into the 4 residual streams

MoE FFN

Hyper-connections mix again to form the input → RMSNorm

Routing: score 384 experts, pick 6, normalize weights, × 1.5
Routed experts: SwiGLU (clamp 10), 5,120 → 2,304 → 5,120
MXFP4, ~18.8 MB per expert, 7.2 GB per layer
1 shared expert: computed for every token
Weighted sum written back to the 4 residual streams



6 of 384 experts chosen

Engram lookup

Only in layers 1 and 14: look up two large tables by n-gram hashes of the last 2–4 tokens
Each ~384M rows × 256 dims; 24 rows/token (3 lengths × 8 heads), gated into the residual stream

Sparse attention indexer (layers 2/8/14/20/24/28/32/36)

Index query

q_a output → wq_b
→ 32 heads × 128 dims
→ RoPE → FP4

Index key

Compressor latent → wk
→ norm → RoPE → FP4
Cached at KV-source layers

Head weights

proj(x) / √(128·32)
One weight per head

Score = Σ_h ReLU(q_h · k) · w_h; pick top-512 compressed rows among positions the query can see
Layer 20: 8 rows per block; picks 2,048 blocks by their highest score (newest block always kept)
Layers 24/28/32/36 reselect only in these blocks; others reuse the nearest index source's top-512

Plan B storage layout (this machine)

GPU0 · RTX PRO 6000 Max-Q, 95.6 GiB

Layers 0–11 in full (experts 80.6 GiB + dense), layer 12 gate/up experts, first half of the head

GPU1 · RTX PRO 6000 Max-Q, 95.6 GiB

Dense part of layers 12–39; experts of layers 12–21 (L12: down only); second half of the head
Expert cache 21.1 GB: hot experts of layers 22–39, ~1,170 slots

Host memory · 192 GiB DDR5 (pinned)

Experts of layers 22–39, 121 GiB
Decode cache misses: hot ones uploaded into the cache, the rest on the CPU, added a layer later

NVMe · Samsung 9100 PRO

Two Engram tables, ~97 GiB (MXFP4), read via the page cache; whole batch prefetched before prefill

Key: KV src writes compressed KV · Idx src picks top-512 · Blocks · Engram

Sources: GGUF metadata (mxxm-t/DeepSeek-V4.1-Flash-GGUF), the official inference/model.py; layout and measurements on this machine: see the technical report and CLAUDE.md (2026-11-11)